KRISTIANSTAD UNIVERSITY

Sweetheat Kök by Bushra

Lab 2 – Interactive Portfolio Enhancements

Project Report

A personal cooking portfolio combining Bangladeshi flavours, Asian heritage and European inspiration.

Student name:	Bushra
Student ID:	[Add your student ID]
Course/Lab:	Lab 2 – Interactive Portfolio Enhancements
Repository:	https://github.com/BushraJannat/Sweetheat-Kok-by-Bushra

2 November 2026

Contents

1 Project Overview **2**

2 Technologies and Project Structure **2**

3 Implemented Features **2**
3.1 Newsletter Subscription Demonstration 2
3.2 Interactive Feedback Form 2
3.3 Dynamic Cooking Skills Showcase 2
3.4 Accessibility Display Settings 3

4 Screenshot Evidence **3**

5 Code Explanation **6**
5.1 Feedback Form: Events, Validation and DOM Updates 6
5.2 Skills: Data-Driven Rendering, Filtering and Sorting 7
5.3 Accessibility: Applying and Saving Preferences 7

6 Problem Solving and Accessibility Considerations **7**

7 Testing **7**

8 Reflection **8**

9 Conclusion **8**

1 Project Overview

Sweetheat Kök by Bushra is a personal cooking portfolio that presents my cooking journey, recipes, food gallery and practical cooking skills. The visual identity is inspired by Bangladeshi flavours, Asian heritage and European influences. The purpose of Lab 2 was to enhance the existing portfolio with interactive features using HTML, CSS and vanilla JavaScript.

The project uses client-side JavaScript. It does not require a server or backend to run the portfolio locally. The newsletter form is a front-end demonstration: it validates the entered email address and displays feedback, but it does not send or store an actual email subscription.

2 Technologies and Project Structure

- **HTML5:** page structure, forms, labels and semantic content.
- **CSS3:** layout, visual styling, responsive presentation and accessibility states.
- **Vanilla JavaScript:** form validation, dynamic skill cards, filtering, sorting and accessibility settings.
- **localStorage:** persistence of selected text size and contrast preference in the browser.
- **Git and GitHub:** source control and project history.

The main project files include `index.html`, `journey.html`, `recipes.html`, `gallery.html`, `feedback.html`, `style.css`, `newsletter.js`, `feedback.js`, `skills.js` and `accessibility.js`.

3 Implemented Features

3.1 Newsletter Subscription Demonstration

The homepage contains a newsletter subscription form. JavaScript checks that the email field contains a valid email address and displays a message to the visitor. This is a demonstration only; no email is sent because the project has no mailing-list backend.

3.2 Interactive Feedback Form

The feedback form collects a name, email address, topic, comments and an optional website. The JavaScript validates the required fields, checks the email format, requires at least 15 characters in the comments field and validates the optional website when it is provided. The character counter updates while the visitor types. The form also provides validation feedback, a Clear button and a success message after valid submission. After successful submission, the form fields and validation states are reset.

3.3 Dynamic Cooking Skills Showcase

The skills page uses a JavaScript array containing eight skill objects. Each object includes a name, category, proficiency level, years of experience and description. The page creates skill cards dynamically from this data. Visitors can filter skills by category and sort them by proficiency, experience or name. The result counter updates to show how many skills are displayed.

3.4 Accessibility Display Settings

The accessibility panel provides Small, Medium and Large text settings and a High Contrast toggle. JavaScript updates the page classes and the accessible state of the controls. The selected text size and contrast preference are saved in `localStorage`, allowing the browser to restore them after a refresh.

4 Screenshot Evidence

The following figures use screenshots captured from the project in the browser. Captions describe what is visibly shown in each image.

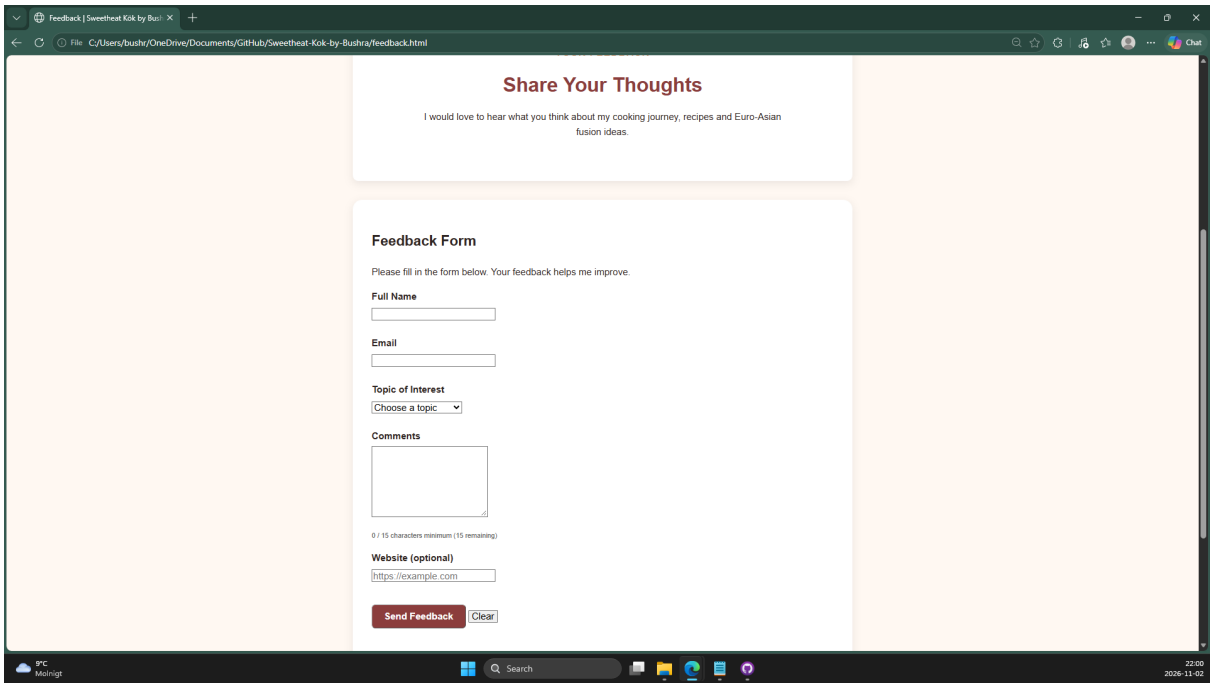


Figure 1: Feedback form in its normal state, with empty fields and the character counter showing the 15-character minimum.

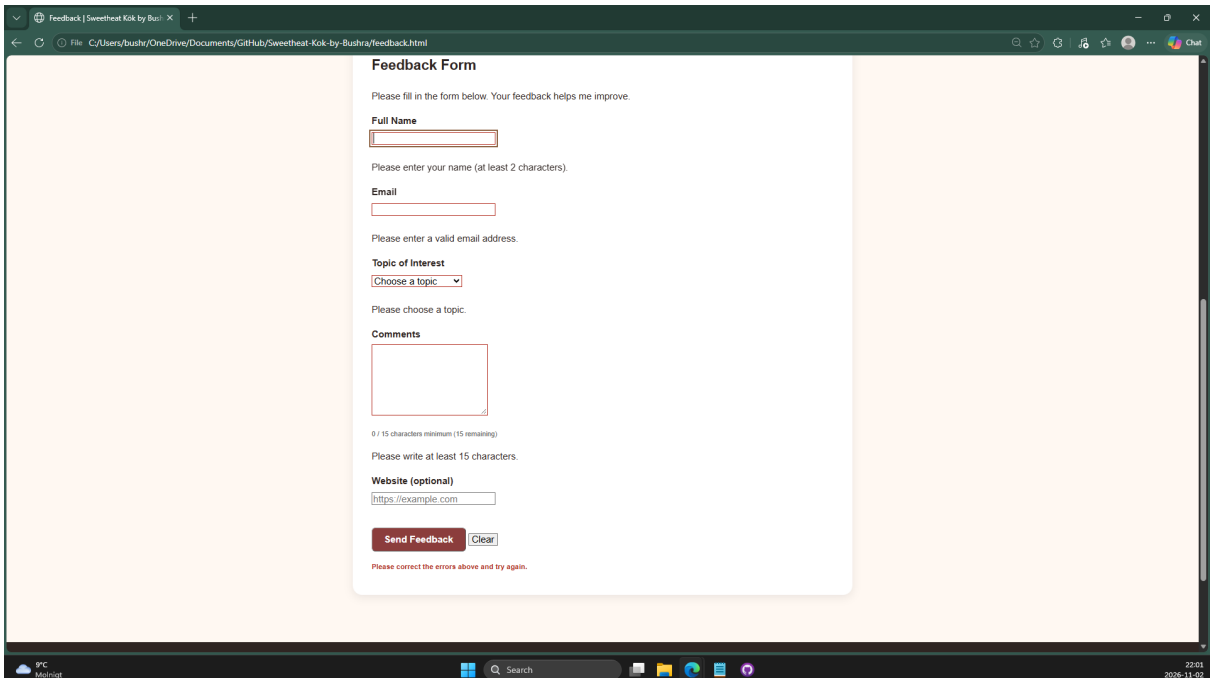


Figure 2: Feedback form displaying validation errors after an empty submission.

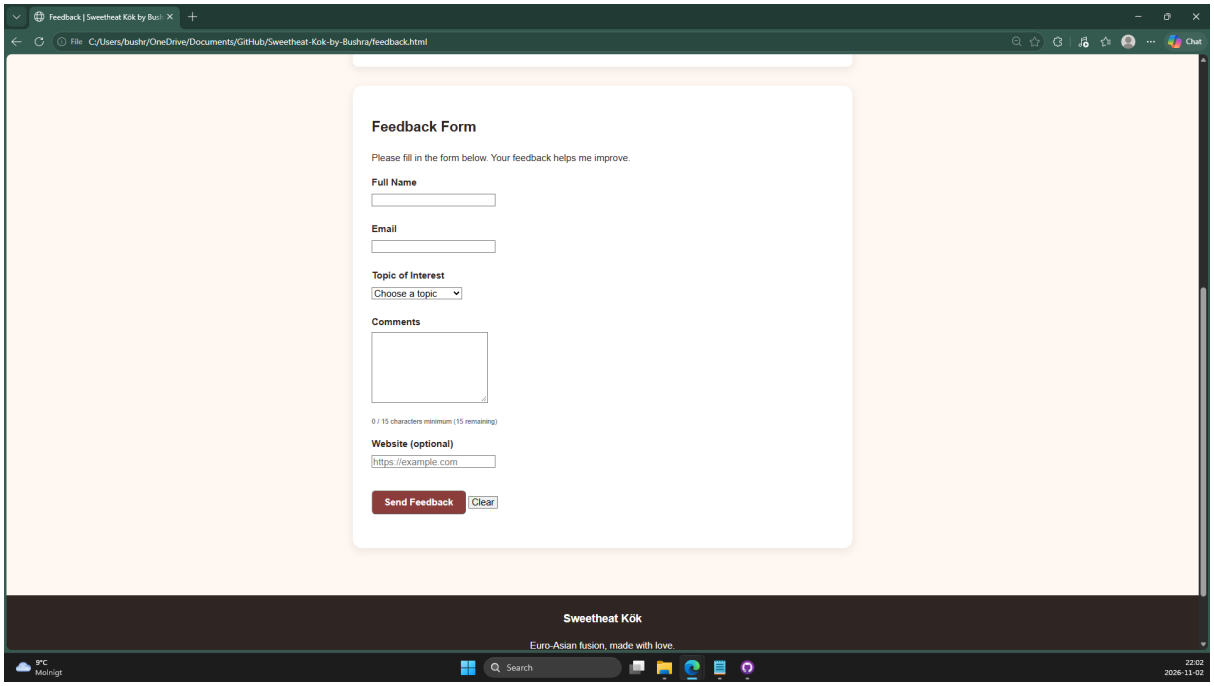


Figure 3: Feedback form showing the cleared state, with fields reset and no validation messages visible.

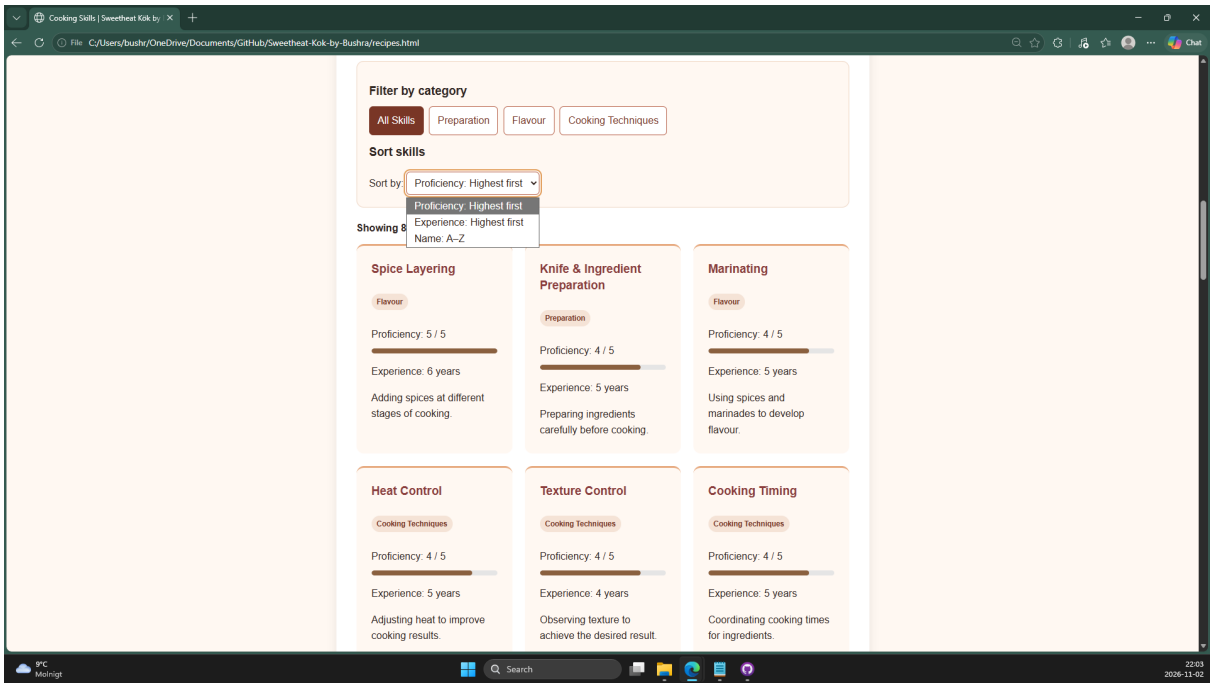


Figure 4: Cooking skills page showing category filter controls, the proficiency sorting menu and dynamically rendered skill cards.

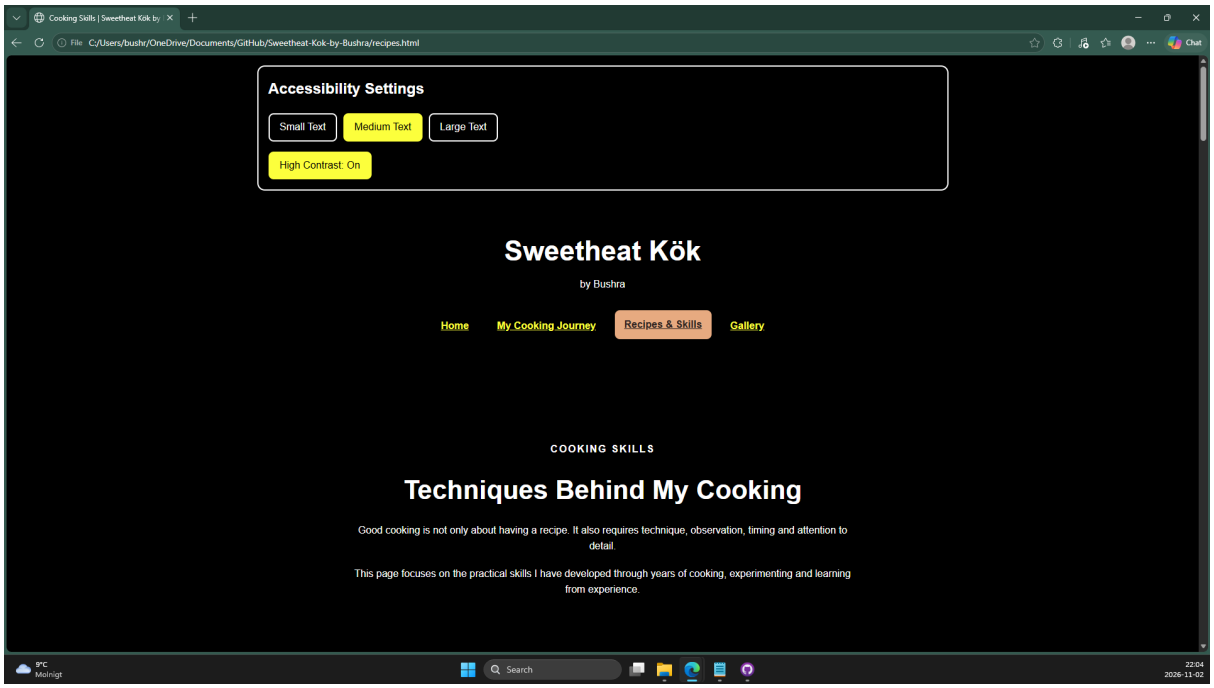


Figure 5: Accessibility panel and portfolio page with Medium Text selected and High Contrast enabled.

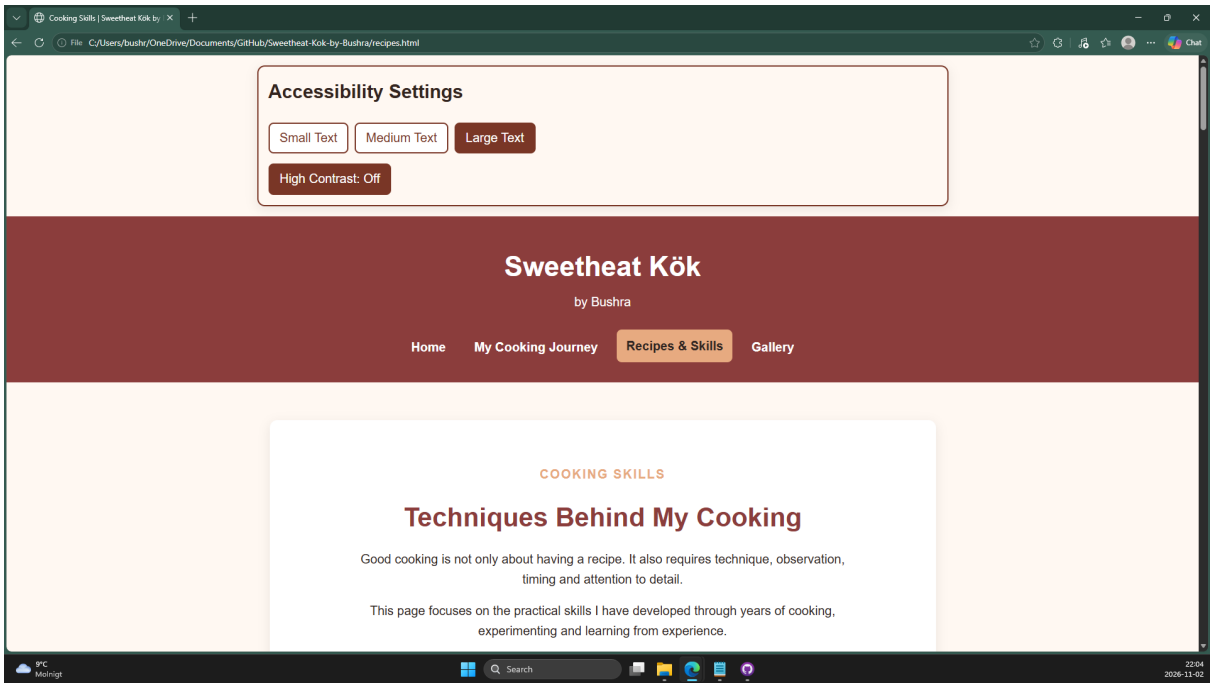


Figure 6: Accessibility panel and portfolio page with Large Text selected and High Contrast disabled.

5 Code Explanation

5.1 Feedback Form: Events, Validation and DOM Updates

The feedback script selects the form elements using `getElementById`. It attaches `input` and `change` event listeners to the fields so that validation can respond while the visitor types or changes a selection. The validation functions return `true` or `false`. They update the page by adding or removing CSS classes, setting `aria-invalid`, and writing messages into the related error elements.

When the form is submitted, the submit event is prevented from performing a normal page reload. The script validates all fields. If any field is invalid, it displays a general error message and focuses the first invalid field. If all fields are valid, it displays a personalised confirmation, resets the form and clears the validation states. A timeout removes the confirmation message after approximately four seconds.

The character counter reads the current comments length and updates its text content. This gives immediate feedback about the minimum comment length.

Listing 1: Example of the feedback form submission flow

```
form.addEventListener("submit", function (event) {
    event.preventDefault();

    const nameIsValid = validateName();
    const emailIsValid = validateEmail();
    const topicIsValid = validateTopic();
    const commentsAreValid = validateComments();
    const websiteIsValid = validateWebsite();

    if (!nameIsValid || !emailIsValid || !topicIsValid ||
        !commentsAreValid || !websiteIsValid) {
        statusMessage.textContent =
            "Please correct the errors above and try again.";
```

```
        return;
    }

    const submittedName = nameInput.value.trim();
    statusMessage.textContent =
        "Thanks for your feedback, " + submittedName + "!";
    form.reset();
});
```

5.2 Skills: Data-Driven Rendering, Filtering and Sorting

The skills are stored as objects in an array. The `renderSkills()` function first filters the array using the selected category. It then sorts a copy of the filtered array, so the original data order is not changed. The function clears the previous cards and creates new cards for the visible skills. The counter is updated after rendering.

Each proficiency bar is created dynamically. Its width is calculated from the proficiency value as a percentage of five. ARIA progress-bar attributes communicate the minimum, maximum and current value to assistive technology.

5.3 Accessibility: Applying and Saving Preferences

The `applySettings()` function removes previous text-size and contrast classes, then applies the selected settings to the document body. It updates the `aria-pressed` state of the buttons and saves the preferences in `localStorage`. On page load, the script reads the saved values and applies them. The text-size value is checked against the allowed options, with Medium used as the fallback.

6 Problem Solving and Accessibility Considerations

One important implementation challenge is keeping the visual validation state and the accessible state consistent. The feedback form addresses this by updating the error message, CSS class and `aria-invalid` attribute together. The error text is associated with its input through `aria-describedby` in the HTML.

For the skills page, the filtering and sorting operations are performed in sequence. Filtering first ensures that sorting applies only to the currently selected category. A copied array is sorted to avoid changing the original skills data.

For accessibility settings, saved values need to be checked before being applied. The script accepts only the supported text-size values and uses Medium when the stored value is not valid. The interface also updates button states so that the selected settings are communicated to assistive technology.

7 Testing

The project should be checked in the browser using the following test cases. Mark each item after completing the test on the final project version.

– Newsletter form: test an invalid email and a valid email; confirm that the message is a demo and no email is sent.

- Feedback form: test empty fields, invalid email, missing topic, short comments and an invalid optional website.
- Feedback form: submit valid information and check the exact personalised success message, reset behaviour and message timeout.
- Feedback form: use the Clear button and confirm that values, errors, counter and status return to the initial state.
- Skills: confirm that eight skills and all three categories are available.
- Skills: test each category filter and each sorting option; test a filter and sort together.
- Accessibility: test Small, Medium and Large text with both Normal and High Contrast modes (six combinations).
- Accessibility: refresh the page and confirm that the selected settings are restored.
- Keyboard: navigate controls using Tab and activate them using Enter and Space.
- Responsive layout: inspect the pages on desktop and mobile screen sizes.
- Developer tools: check that the browser console has no JavaScript errors.

The screenshots in this report document the visible states captured during testing. The final test checklist should be completed against the submitted version of the website.

8 Reflection

This project helped me practise connecting JavaScript to HTML elements through events and DOM updates. I learned how to validate form input, provide immediate feedback and reset a form after submission. I also gained experience with arrays of objects, dynamically generated elements, filtering and sorting data.

The accessibility work showed me that an interface should communicate its state to more than just sighted users. ARIA attributes, keyboard interaction, readable text sizes and persistent preferences all contribute to a more usable experience. I also learned that testing should include different input states, keyboard use, refresh behaviour and different screen sizes.

9 Conclusion

Lab 2 extended Sweetheat Kök by Bushra from a static cooking portfolio into a more interactive website. The project includes a newsletter demonstration, a validated feedback form, a data-driven cooking skills showcase and accessibility display settings. The work uses HTML, CSS and vanilla JavaScript, with localStorage for accessibility preferences. The final review should confirm all required test cases, screenshots, documentation and repository history before submission.

GitHub repository: `https://github.com/BushraJannat/Sweetheat-Kok-by-Bushra`